# Root-Cause Analysis

*timeline, what changed, narrow, test*

Watch the 4-min video

**In one line:** to debug a failure you've never seen, build a timeline, ask what changed, narrow the scope until the problem is small, and test one hypothesis at a time while writing it down; afterwards, ask "why" repeatedly and fix every contributing factor, not just the trigger.

## 1 No runbook, and the clock is running

Checkout errors jump from almost zero to **5%**. Nobody has seen this failure. There is no runbook, and every minute costs money. Random guessing wastes the one thing you can't get back during an incident: time. A method beats intuition here, because it works on problems you've never seen.

## 2 Timeline first, then: what changed?

When exactly did it start? Sudden or gradual? Then the most useful question in debugging: **what changed?** Deploys, config changes, feature flags, traffic shifts, dependency upgrades, certificate expiries. Most incidents start within minutes of a change, so line up the change log against the moment errors began. A deploy history and a config audit log turn this from an hour into seconds.

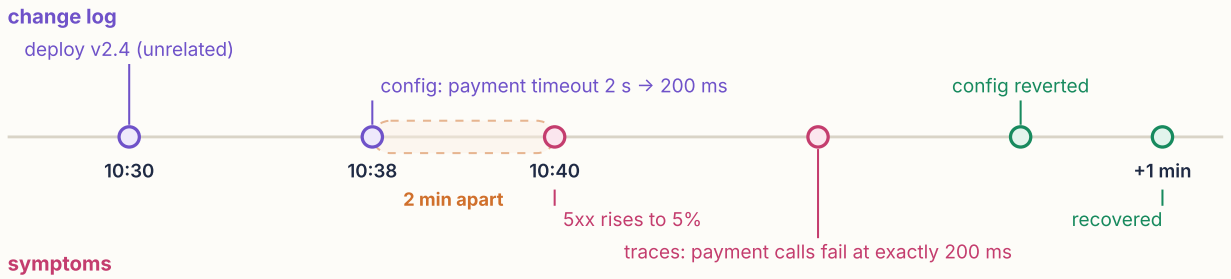



**Figure 1.** The video's incident, with an unrelated 10:30 deploy added to show the point: the change just before the symptom is the first suspect, not proof.

## 3 Narrow the scope

| Ask | If the answer is "only one"... | Where to look |
|---|---|---|
| All endpoints or one route? | code path or dependency of that route | RED dashboard per route (lesson 61) |
| All regions or one? | regional infra, config or traffic | metrics split by region |
| All tenants or one? | that tenant's data, plan or limits | logs by tenant_id (lesson 60) |
| All pods or one node? | bad host, noisy neighbour, disk | metrics by pod and node |
| Which call fails? | a specific dependency | traces of failed requests (lesson 63) |

Each answer cuts the search space roughly in half. Dashboards show where errors concentrate, traces show which call fails, logs show the exact error.

## 4 Hypotheses, one at a time

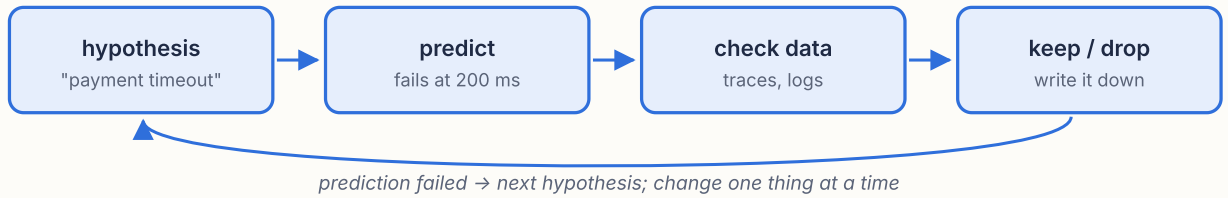



**Figure 2.** The scientific loop. Post each step in the incident channel so nobody repeats a check that has already been done.

State one possible cause, **predict** what you'd see if it were true, and check. If the prediction fails, drop it. Don't change several things at once: if errors stop, you won't know which change helped, and one of them may cause the next incident. During the incident, mitigation can come before certainty: reverting the suspect change is often both the test and the fix (lesson 70).

## 5 Five whys and contributing factors

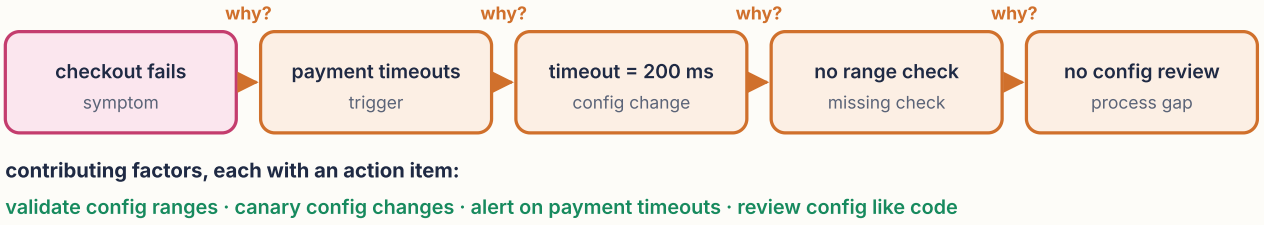



**Figure 3.** Each "why" goes one level deeper, from symptom to something fixable. Real incidents have several contributing factors, not one root cause.

Stopping at the trigger ("someone set 200 ms") leaves the system just as fragile and invites blame. Keep asking until you reach system properties: missing validation, no canary for config, an alert that didn't fire. Assign an action item to each factor. The goal is a system that is harder to break, not a name in a report.

**WORKED EXAMPLE: ELEVATED 5XX ON CHECKOUT**

At **10:40** checkout returns elevated 500s. The timeline shows a config change at **10:38** lowering the payment client's timeout from **2 s to 200 ms**. Hypothesis: payments time out. Prediction: failed payment spans end at exactly 200 ms. Traces confirm it. The team reverts the config; errors drop within a minute. Action items: validate config ranges, canary config changes like code, alert on payment timeouts.

**COMMON MISTAKES**

- Debugging the code before checking the change log.
- Changing three things at once, then not knowing which one fixed it.
- Stopping at "human error": that's where the analysis should start.
- Treating correlation as proof: the deploy at 10:30 also preceded the errors.

## 6 Key terms

**Timeline**: ordered record of changes, symptoms and actions

**Change log**: deploys, config and flag changes with timestamps

**Hypothesis**: a testable guess with a predicted observation

**Trigger**: the event that set the failure off

**Five whys**: asking why repeatedly to reach fixable causes

**Contributing factor**: a condition that let the trigger cause harm

## 7 Interview questions

**Q1. How do you investigate a production incident you've never seen before?** BASIC

Build a timeline, check what changed (deploys, config, flags, traffic), narrow the scope with dashboards, traces and logs, then test hypotheses one at a time, writing every step down.

**Q2. What is the five whys?** BASIC

Asking why repeatedly, from the symptom down through the trigger to fixable causes and contributing factors, each of which gets an action item.

**Q3. Errors started at 14:02; there was a deploy at 13:10 and a config change at 14:00. Where do you start?** SCENARIO

With the 14:00 config change: it's closest in time. Predict what it would cause, check traces or logs, and revert it if it fits.

**Q4. Errors affect only one tenant. What does that suggest?** SCENARIO

Something specific to that tenant: its data, plan limits, a tenant-specific flag or a noisy-neighbour quota. Filter logs and traces by tenant_id.

**Q5. Why is "one root cause" a misleading idea?** TRICKY

Failures need several conditions at once: a risky change, a missing check, an alert that didn't fire. Fixing only the trigger leaves the other gaps open.

**Q6. Should you revert before you're sure the change caused it?** TRICKY

Often yes, if the revert is low-risk: it limits impact and doubles as a test. If errors stop, you've confirmed the hypothesis; if not, you've ruled it out.

Next: SD-N-012 · Incident Management backendroadmap.com